MODELO COMPLETO DE PROJETO DE PROGRAMAÇÃO WEB

Projeto: MaisonHuambo - Plataforma de Gestão Imobiliária

---

1. Informações Gerais

- **Nome do Projeto:** MaisonHuambo - Plataforma de Gestão Imobiliária
- **Curso / Disciplina:** Engenharia Informática / Programação Web
- **Professor:** Yandilier Rojas Lorenzo
- **Data de entrega:** 06 de Abril de 2026

Equipe de Trabalho

Para organização das tarefas, a equipe está estruturada da seguinte forma:

- **Paulo Kapitia** (Líder da Equipe)
- **Raimundo Sonjamba**

---

2. Descrição do Projeto

O **MaisonHuambo** é um sistema web responsivo para a gestão e divulgação de imóveis, com foco na automação de processos para corretores e clientes. O sistema atua como um catálogo digital de propriedades, permitindo pesquisa com filtros, solicitações de visita e um painel de administração seguro para a gestão dos anúncios publicados.

---

3. Justificativa

- **Problema que resolve:** O mercado imobiliário local sofre frequentemente com a desorganização da informação e a dificuldade em coordenar visitas entre múltiplos clientes e corretores, resultando em sobreposição de horários.
- **Benefícios esperados:** Centralização dos dados imobiliários, redução do tempo gasto em comunicações manuais para agendamento e maior transparência nos preços e detalhes dos imóveis.
- **Impacto acadêmico, social ou empresarial:**
  - *Acadêmico:* Consolida a proficiência na construção de APIs e manipulação do DOM.
  - *Empresarial:* Propõe um modelo digitalizável para pequenas agências imobiliárias.

---

4. Objetivos

Objetivo Geral

Desenvolver um sistema web funcional ponta a ponta para o setor imobiliário.

Objetivos Específicos

- Projetar interface fluida e responsiva.
- Implementar banco de dados relacional com integridade.
- Desenvolver backend escalável em Node.js.
- Integrar frontend e backend via requisições HTTP (Fetch API).
- Realizar testes de rotas da API e usabilidade.

---

5. Escopo

- **Inclui:** Autenticação de corretores (login), CRUD (Criar, Ler, Atualizar, Apagar) de imóveis, upload de imagens, filtros dinâmicos de pesquisa, e módulo de agendamento de visitas.
- **Não inclui:** Processamento de pagamentos online (gateways financeiros), emissão de faturas com validade fiscal ou assinatura digital de contratos.

---

6. Tecnologias

- **Frontend:** HTML, CSS e JavaScript puro (Vanilla JS).
- **Backend:** Node.js (com Express.js).
- **Banco de Dados:** PostgreSQL (via Supabase).
- **Ferramentas:** Git, GitHub (fluxo remoto com revisão de código e merges centralizados), VS Code.

---

7. Requisitos

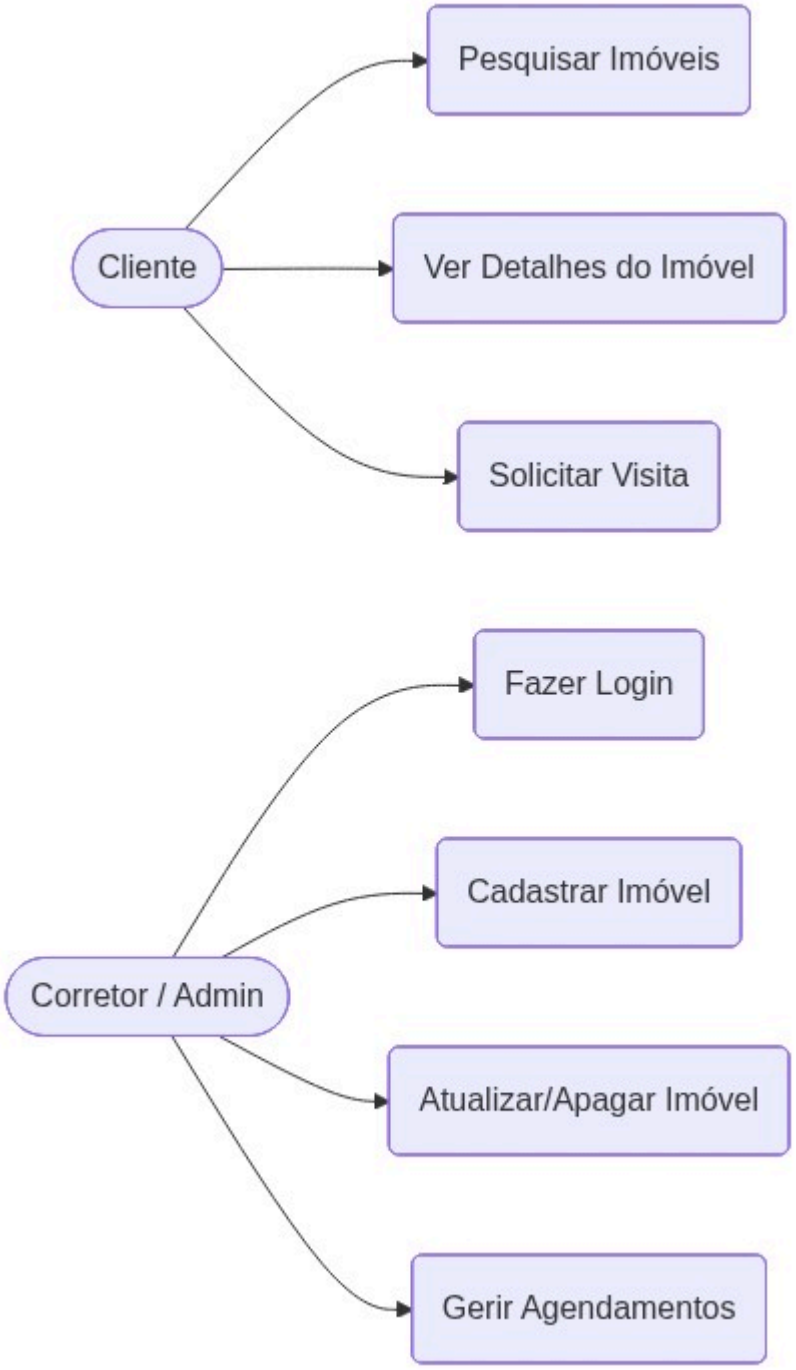
- **Funcionais:** Cadastro, login, CRUD (gestão de imóveis) e relatórios (agendamentos).
- **Não funcionais:** Segurança (proteção de rotas e dados de login), desempenho (carregamento rápido das imagens do catálogo) e usabilidade (interface adaptável a dispositivos móveis e navegação intuitiva).

---

8. Modelagem UML

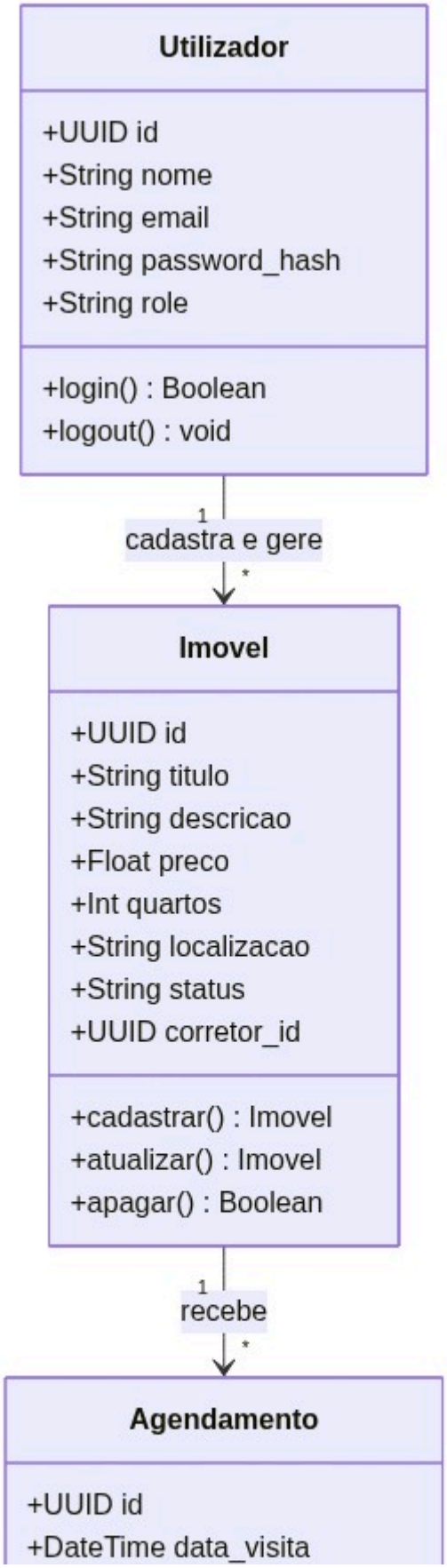
A. Diagrama de Casos de Uso

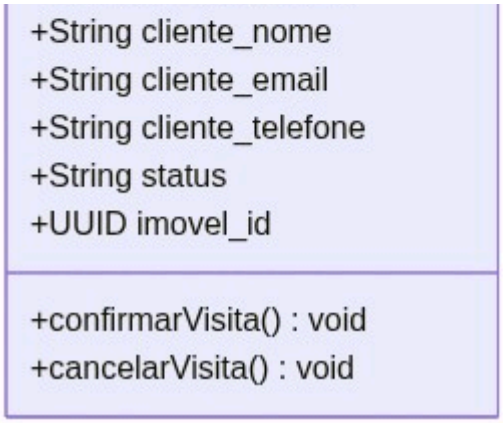
Interações do cliente e corretor com o sistema.



B. Diagrama de Classes

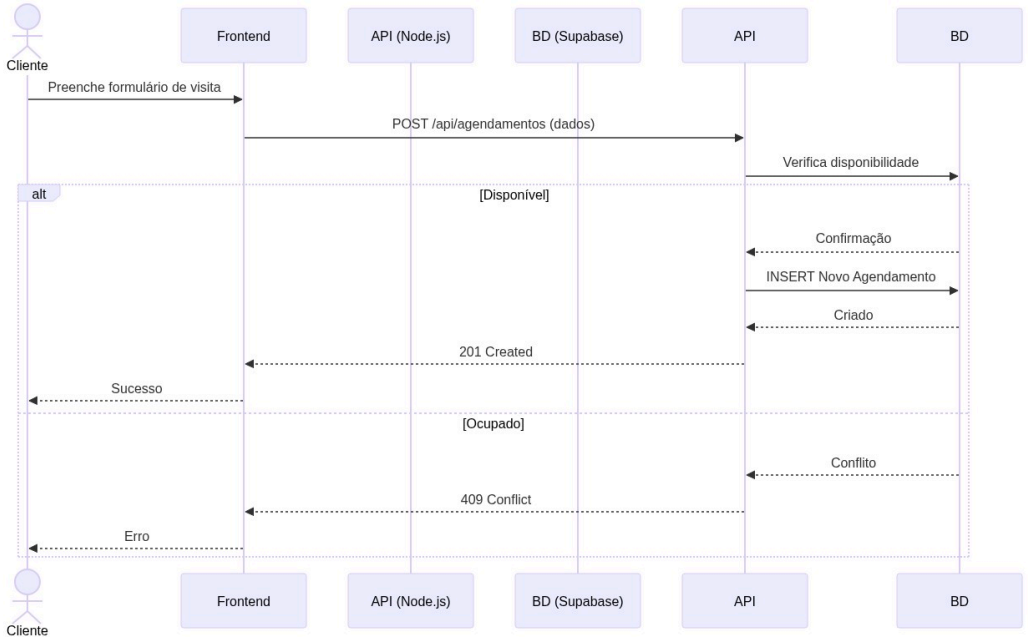
Estrutura das entidades do domínio.


Utilizador
+UUID id
+String nome
+String email
+String password_hash
+String role
+login() : Boolean
+logout() : void
1
cadastra e gere
*
Imovel
+UUID id
+String titulo
+String descricao
+Float preco
+Int quartos
+String localizacao
+String status
+UUID corretor_id
+cadastrar() : Imovel
+atualizar() : Imovel
+apagar() : Boolean
1
recebe
*
Agendamento
+UUID id
+DateTime data_visita



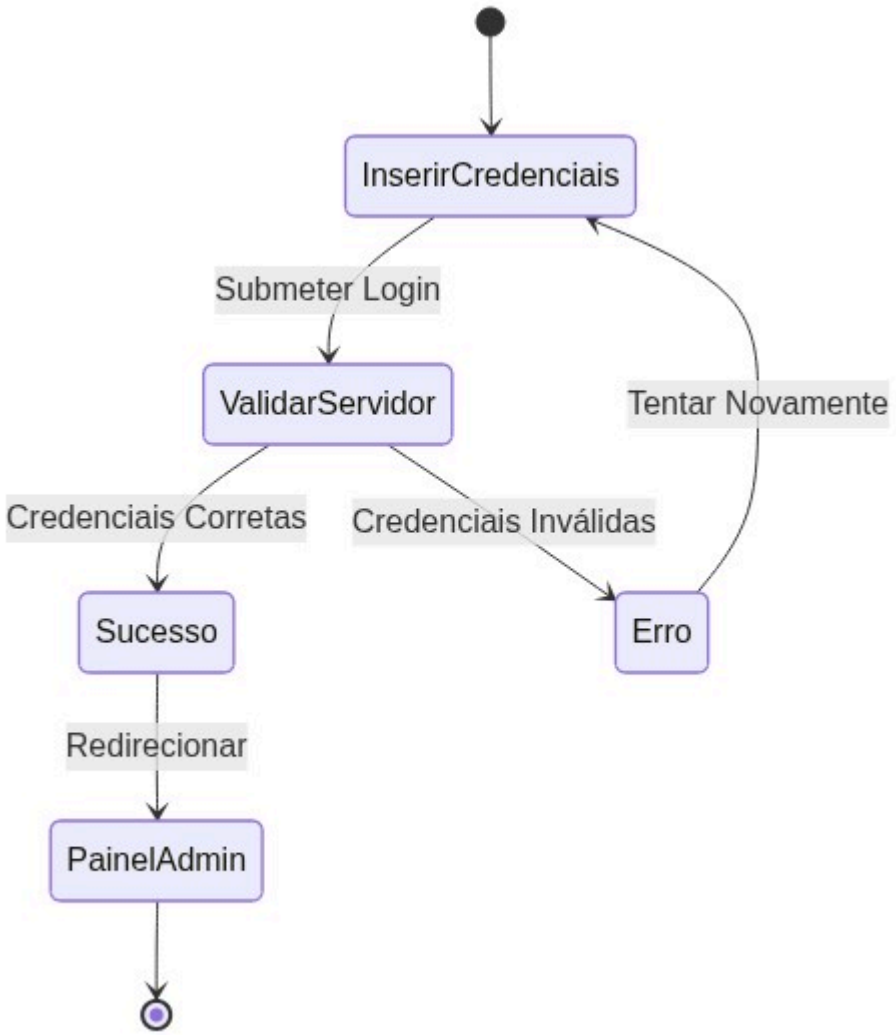
C. Diagrama de Sequência

Fluxo cronológico de uma solicitação de visita.



D. Diagrama de Atividades

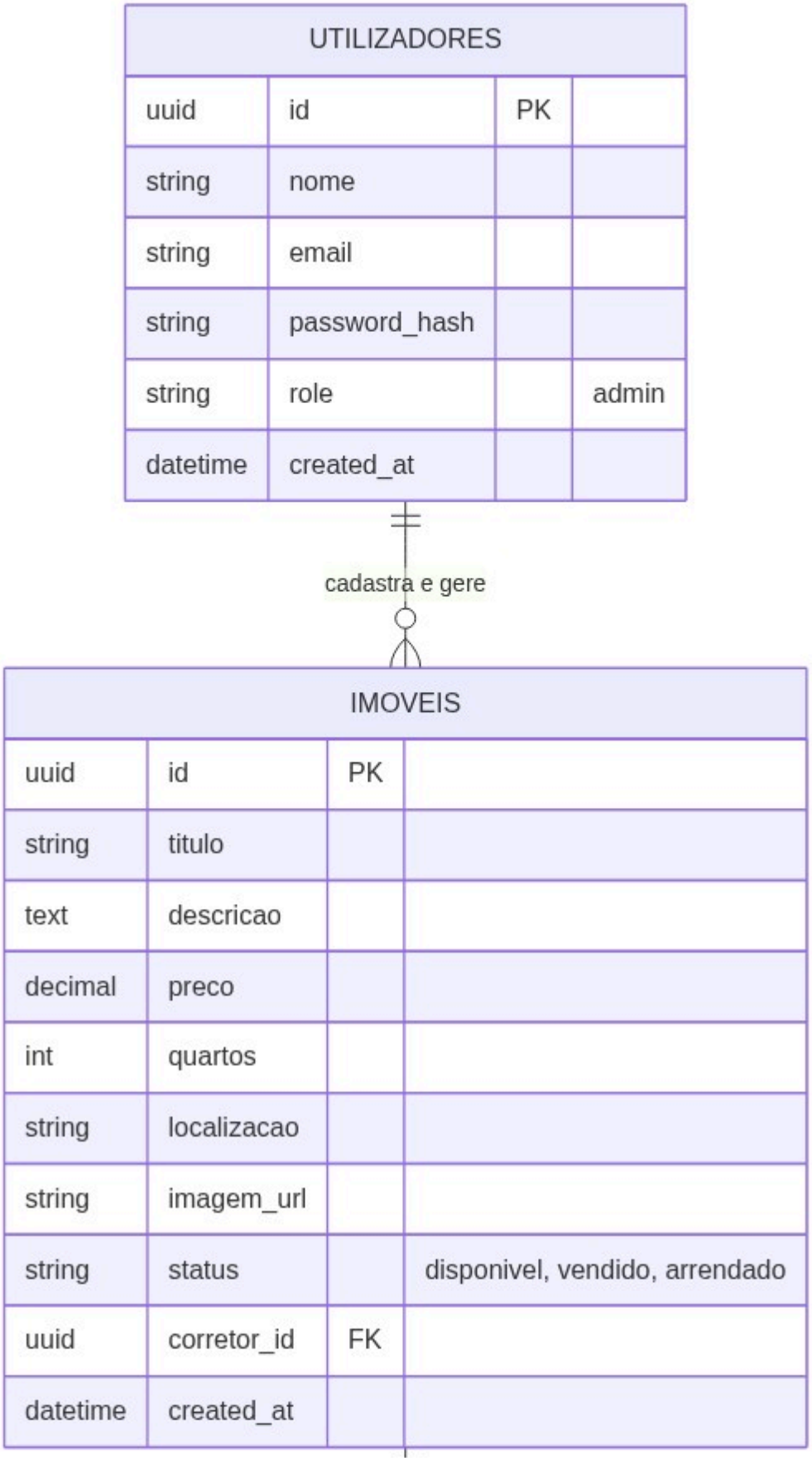
Lógica do processo de autenticação.

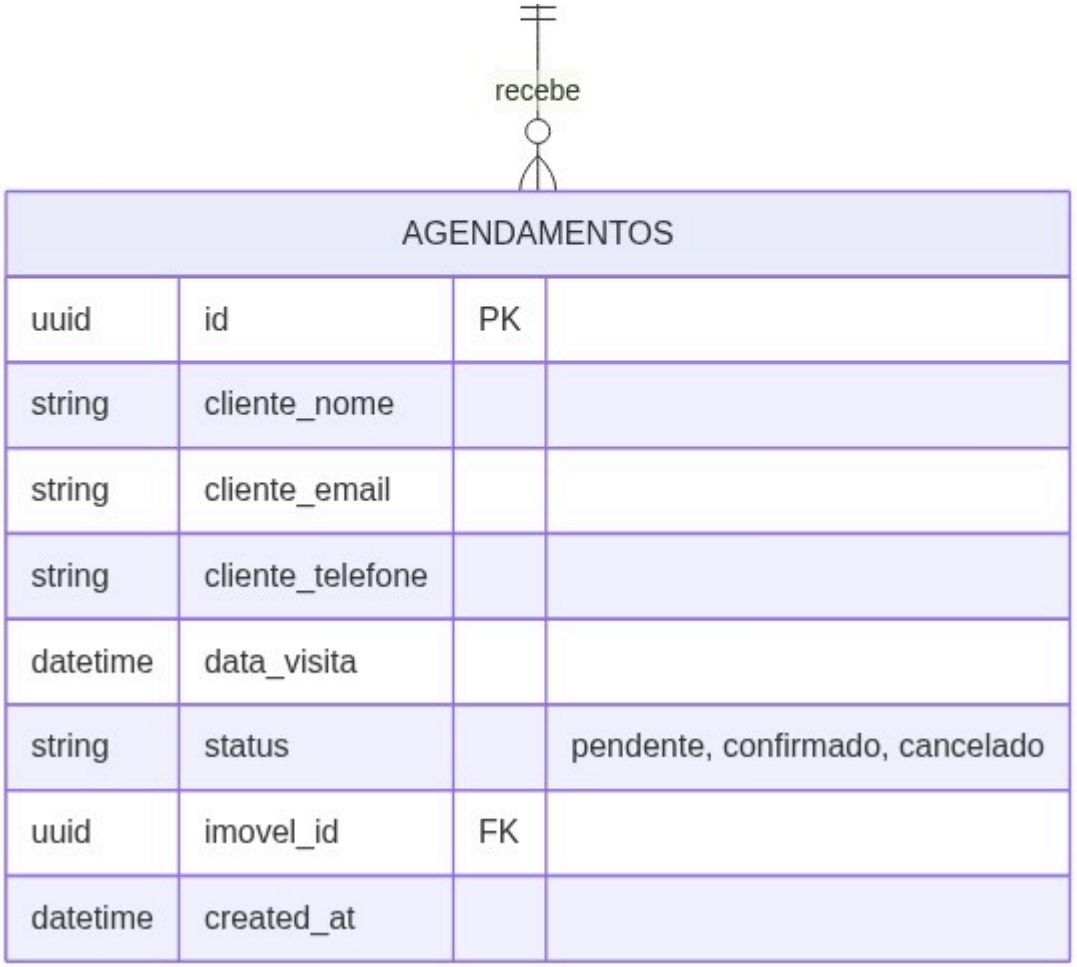


---

9. Banco de Dados

Modelo Entidade-Relacionamento (ERD)


UTILIZADORES
uuid
id
PK
string
nome
string
email
string
password_hash
string
role
admin
datetime
created_at
cadastra e gere
IMOVEIS
uuid
id
PK
string
titulo
text
descricao
decimal
preco
int
quartos
string
localizacao
string
imagem_url
string
status
disponivel, vendido, arrendado
uuid
corretor_id
FK
datetime
created_at



- **Tabela Utilizadores:** Dados de acesso dos corretores.
- **Tabela Imoveis:** Catálogo principal com relação 1:N com Utilizadores.
- **Tabela Agendamentos:** Pedidos de visita relacionados a cada imóvel.

---

10. Arquitetura

- **Cliente-Servidor:** Desacoplamento entre frontend e backend.
- **MVC:** Organização lógica do servidor (Models, Views, Controllers).
- **API REST:** Comunicação eficiente via JSON.

Estrutura de Diretórios

```
MaisonHuambo/
├── backend/                # Código-fonte da API Node.js
│   ├── src/
│   │   ├── controllers/    # Controladores (Lógica de Negócio)
│   │   ├── models/         # Interação com BD (Supabase)
│   │   ├── routes/         # Rotas da API RESTful
│   │   └── server.js       # Configuração e entrada principal
│   ├── .env                # Variáveis de ambiente
│   └── package.json        # Dependências do backend
│
├── frontend/               # Interface do Utilizador (Vanilla JS)
```

```
│   ├── css/                # Folhas de estilo
│   ├── js/                 # Lógica assíncrona (Fetch API, DOM)
│   ├── assets/             # Imagens e ícones
│   ├── index.html          # Landing page e catálogo
│   └── admin.html          # Painel do corretor
│
└── README.md               # Documentação e setup do projeto
```

---

11. Planejamento do Projeto

Abaixo, o cronograma macro estipulado para a construção do projeto do conceito à implantação:

Semana	Fase	Atividades Detalhadas
1	**Análise e Definição**	Levantamento dos requisitos e padronização da modelagem UML (Casos de Uso, Classes, Sequência e Atividades) em conjunto com a modelagem do Banco de Dados (ERD).
2	**Design da Interface**	Esboço de telas, definição do sistema de cores/tipografia e estruturação completa do layout base usando HTML5 e CSS3 puro.
3	**Desenvolvimento Backend**	Instalação do Node.js, configuração do Express.js, definição da arquitetura interna e modelagem das tabelas e chaves estrangeiras no Supabase.
4	**Desenvolvimento Frontend**	Implementação dos formulários, integração via Fetch API, lógica JavaScript para envio e listagem das propriedades cadastradas.
5	**Integração de Regras**	Finalização do módulo de agendamento de visitas com verificação de disponibilidade, controle da lógica de login de corretores e integração final dos módulos.
6	**Testes e Validação**	Bateria de testes nos endpoints da API, correções de estilos em resoluções menores (responsividade em mobile) e mitigação de erros assíncronos.
7	**Entrega e Implantação**	Elaboração da versão final do documento acadêmico, estruturação do manual do usuário e implantação do sistema.

- **Metodologia:** Desenvolvimento colaborativo remoto com fluxo Git-flow (branching/merging via GitHub).

---

12. Orçamento

- **Hospedagem:** 0,00 Kz (Plataform as a Service - Vercel)
- **Domínio:** 0,00 Kz (Subdomínio gratuito Vercel)

- **Ferramentas:** 0,00 Kz (Software Open Source)
- **Total: 0,00 Kz**

---

13. Gestão de Riscos

1. **Falta de tempo:** Mitigação através do foco no MVP (Minimum Viable Product).
2. **Problemas de coordenação:** Mitigação através de revisões semanais da equipe.
3. **Problemas técnicos:** Mitigação através de testes antecipados e validação da API.

---

14. Testes

- **Testes Unitários:** Validação de funções lógicas isoladas.
- **Testes Funcionais (E2E):** Validação dos endpoints da API e fluxos completos de navegação.
- **Testes de Usabilidade:** Feedback sobre a experiência do usuário final.

---

15. Resultados Esperados

- **Sistema robusto:** Fluxo de gestão imobiliária operando sem erros críticos.
- **Interface intuitiva:** Design responsivo focado na facilidade de navegação.
- **Objetivos acadêmicos:** Domínio profundo de Node.js, Express e Vanilla JS.

---

16. Conclusões (Perspectivas)

Com a proposta do projeto **MaisonHuambo**, a equipe tem como meta criar uma solução eficaz para a digitalização do mercado imobiliário local. Através da implementação planejada de uma arquitetura robusta com Node.js e Supabase, prevemos atingir todos os objetivos propostos ao entregar uma plataforma fluida e funcional.

- **Aprendizados Esperados:** Consolidação prática da manipulação assíncrona do DOM (Fetch API), estruturação de APIs RESTful escaláveis e a modelagem rigorosa de bancos de dados relacionais.
- **Desafios Previstos (Dificuldades):** Garantir uma sincronização eficiente entre cliente e servidor em solicitações simultâneas e realizar uma boa gestão de conflitos de código durante o desenvolvimento colaborativo no GitHub.
- **Recomendações para Evolução:** Após a conclusão e entrega desta versão inicial, projeta-se considerar a migração do frontend para frameworks de estado modernos (ex: React/Vue) e a possível integração futura com gateways de pagamentos locais (ex: Referência Multicaixa).

---

17. Anexos

Os anexos (código-fonte, capturas de tela e manual do usuário) serão fornecidos junto à entrega final do sistema, não se aplicando nesta fase de proposta do projeto.